EE:8203: DESIGN AND MANAGEMENT OF DATA NETWORKS

Project Report

Group 02:

Kodithuwakku K.K.A.M – EG/2021/4614
Adhikari A.M.D.P – EG/2021/4386

Contents

List of Tables

List of Figures

1 Executives Summary

The development of campus data network system design, its implementation, automation, and monitoring is the main topic of this project for the Faculty of Engineering of the University of Ruhuna. The proposed network connects four departments: Department of Electrical and Information Engineering (DEIE), Department of Civil and Environmental Engineering (DCEE), Department of Mechanical and Manufacturing Engineering (DMME) and Department of Interdisciplinary Studies (DIS). The network is structured to ensure secure communication between departments, at the same time that it does not give too much access to others, centralize management and monitor the network continually.

In GNS3 the Cisco IOS devices are configured with a three-layer hierarchical network architecture: core, distribution, and access. VLANs logically partition departmental, server, management, and native traffic and trunking and inter-VLAN routing enable controlled communication between network segments. Dynamic routing is used with OSPF and Access Control Lists (ACLs) are used to enforce inter-department security policies.

A dedicated Ubuntu 22.04 automation host (VM-AUTO) automates network configuration. Router configuration is automated with python scripts using the Netmiko library, and switch configuration is done with Ansible and the distribution of the software. Parameterized inventories, error handling, logging, repeatable configuration, idempotency, and rollback is part of the automation design.

VM-ZABBIX is installed with Zabbix and SNMPv2c for centralized network monitoring, which monitors all routers and switches. The availability of devices, interface status, CPU utilization, and other network parameters are monitored by setting the proper triggers and alerts. There is also a custom dashboard created to give a single point of view about the network's availability, traffic and active alerts, called a “FoE-UoR Network” dashboard.

Completed implementation is checked and demonstrated through configuration checks, connectivity testing, ACL reachability tests, automation execution, idempotency verification and Zabbix monitoring. Overall, the project showcases the

implementation of a structured campus network that leverages VLAN segmentation, dynamic routing, ACL-based security, network automation, and centralized monitoring, providing a scalable and manageable network infrastructure for future growth and expansion. The project needs state that these parts should be verified live in the evaluation.

2 Network Design

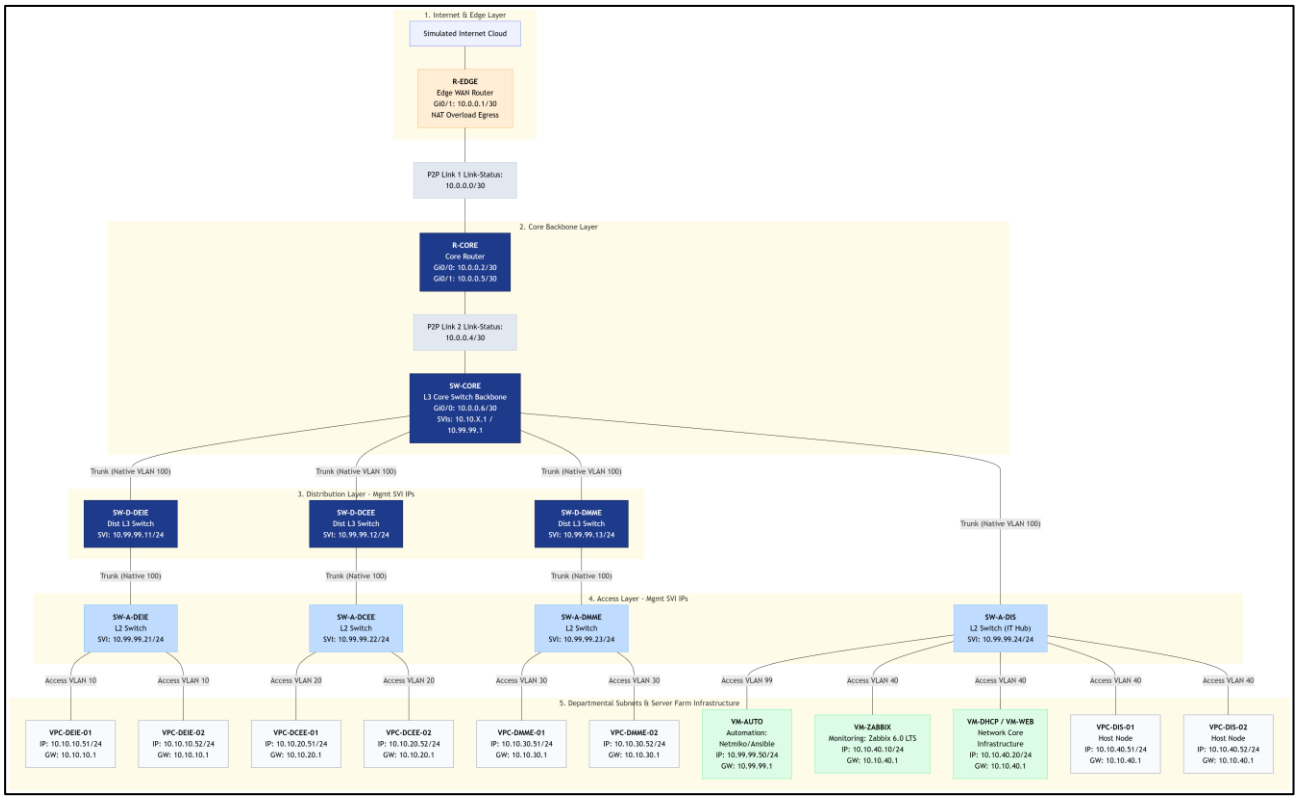

Figure 2-1: Illustrated Complete Network Topology

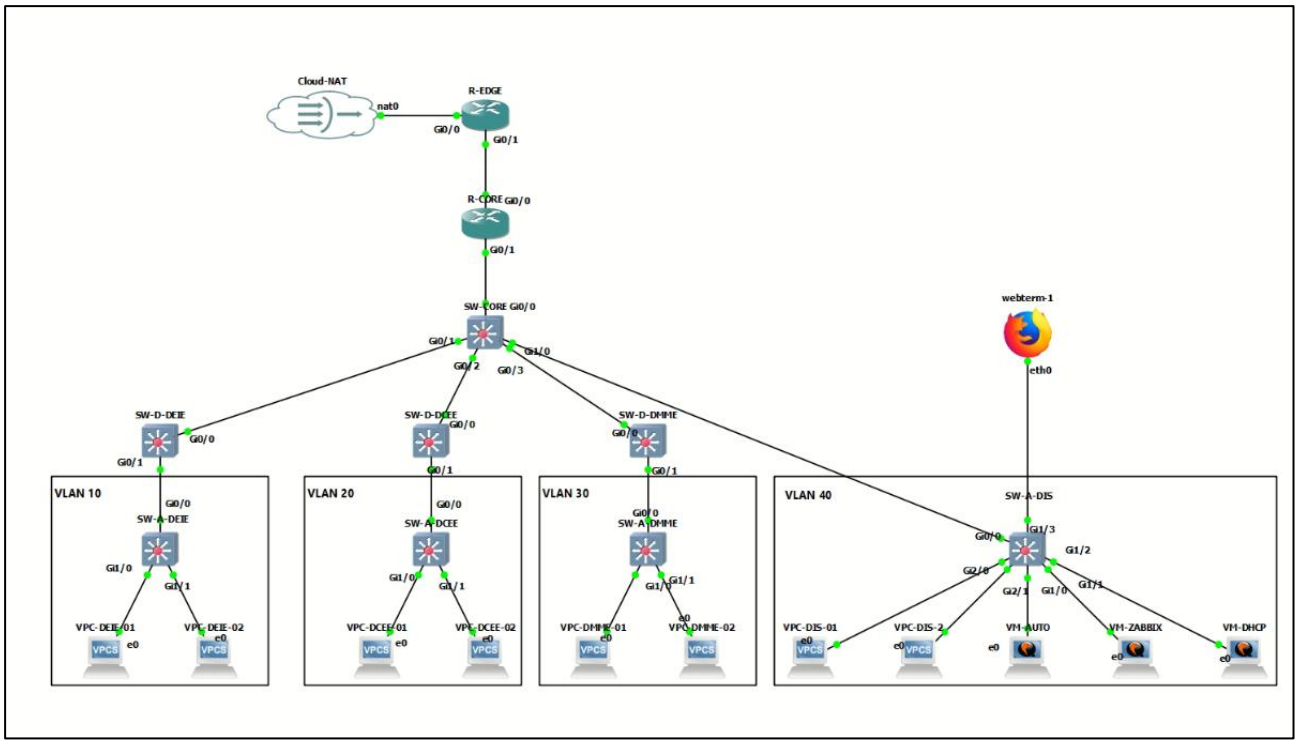

Figure 2-2 : Network Implementation in GNS3

As shown in Figure 2-1, prior to the GNS3 the complete network was designed including all devices, IP addresses, VLANS, SVIs, and ports. Then it was implemented in the GNS3 as shown in Figure 2-2. The network is designed to interconnect the DEIE, DCEE, DMME, and DIS departments while providing logical network segmentation, controlled inter-department communication, centralized management, automation, and monitoring. The project requires the network to follow a three-layer hierarchical architecture consisting of Core, Distribution, and Access.

The architecture is divided into the following layers:

Table 2-1 : Functional Roles of Devices Across the Three-Tier Campus Network Architecture

Layer	Devices	Function
Core Layer	R-CORE, SW-CORE	Provides the main campus network backbone. R-CORE operates as the core router using OSPF Area 0 and NAT, while SW-CORE functions as the core Layer-3 switch and supports inter-VLAN routing.

Distribution Layer	SW-D-DEIE, SW-D-DCEE, SW-D-DMME	Aggregates departmental traffic between the core and access layers. One Layer-3 distribution switch is assigned to each academic department.
Access Layer	SW-A-DEIE, SW-A-DCEE, SW-A-DMME, SW-A-DIS	Provides end-device connectivity. VPC end hosts connect at this layer and are used for network connectivity and ACL testing.

Table 2-2 : VLAN, Network Address, Default Gateway Configurations in the Architecture

VLAN ID	VLAN Name	Department / Purpose	Network Address	Default Gateway
10	VLAN_DEIE	Electrical & Information Engineering	10.10.10.0/24	10.10.10.1
20	VLAN_DCEE	Civil & Environmental Engineering	10.10.20.0/24	10.10.20.1
30	VLAN_DMME	Mechanical & Manufacturing Engineering	10.10.30.0/24	10.10.30.1
40	VLAN_DIS	DIS Server Farm	10.10.40.0/24	10.10.40.1
99	MGMT	Network device management	10.99.99.0/24	10.99.99.1
100	NATIVE	Native VLAN for trunk links	No IP subnet	—

2.1 IP Addressing Plan

Table 2-3 : IP Addressing and Network Interface Allocation

Device	Interface	IP Address	Subnet Mask	Network
R-EDGE	G0/0			NAT network
R-EDGE	G0/1	10.0.0.1	255.255.255.252	10.0.0.0/30
R-CORE	G0/0	10.0.0.2	255.255.255.252	10.0.0.0/30

R-CORE	G0/1	10.0.0.5	255.255.255.252	10.0.0.4/30
SW-CORE	G0/0	10.0.0.6	255.255.255.252	10.0.0.4/30
SW-CORE	VLAN 10	10.10.10.1	255.255.255.0	10.10.10.0/24
SW-CORE	VLAN 20	10.10.20.1	255.255.255.0	10.10.20.0/24
SW-CORE	VLAN 30	10.10.30.1	255.255.255.0	10.10.30.0/24
SW-CORE	VLAN 40	10.10.40.1	255.255.255.0	10.10.40.0/24
SW-CORE	VLAN 99	10.99.99.1	255.255.255.0	10.99.99.0/24

Table 2-4 : Management VLAN and IP Address Assignment for Network Switches

Device	Management Interface	Management IP
SW-CORE	VLAN 99	10.99.99.1
SW-D-DEIE	VLAN 99	10.99.99.11
SW-D-DCEE	VLAN 99	10.99.99.12
SW-D-DMME	VLAN 99	10.99.99.13
SW-A-DEIE	VLAN 99	10.99.99.21
SW-A-DCEE	VLAN 99	10.99.99.22
SW-A-DMME	VLAN 99	10.99.99.23
SW-A-DIS	VLAN 99	10.99.99.24

Table 2-5 : IP Addressing, VLAN Assignment, and Default Gateway Configuration of Departmental and Server-Farm Hosts

Host	VLAN	IP Address	Subnet Mask	Default Gateway
VPC-DEIE-01	10	10.10.10.51	255.255.255.0	10.10.10.1
VPC-DEIE-02	10	10.10.10.52	255.255.255.0	10.10.10.1
VPC-DCEE-01	20	10.10.20.51	255.255.255.0	10.10.20.1
VPC-DCEE-02	20	10.10.20.52	255.255.255.0	10.10.20.1

VPC-DMME-01	30	10.10.30.51	255.255.255.0	10.10.30.1
VPC-DMME-02	30	10.10.30.52	255.255.255.0	10.10.30.1
VPC-DIS-01	40	10.10.40.51	255.255.255.0	10.10.40.1
VPC-DIS-02	40	10.10.40.52	255.255.255.0	10.10.40.1

Table 2-6 : IP Addressing and Default Gateway Configuration of Server-Farm

Device	VLAN	IP Address	Default Gateway
VM-AUTO	99	configured IP	10.99.99.1
VM-ZABBIX	40	10.10.40.10	10.10.40.1
VM-DHCP/WEB	40	10.10.40.20	10.10.40.1

Table 2-7 : Device Justification with main functions

Device	Layer / Role	Main Function	Reason for Selection
R-CORE	Core	OSPF Area 0, Core routing	Provides core routing between major network segments
R-EDGE	Edge/WAN	Default route, NAT overload	Provides external/WAN connectivity and Internet address translation
SW-CORE	Core L3	Inter-VLAN routing backbone	Provides high-speed Layer-3 switching and central campus connectivity
SW-D-DEIE	Distribution	DEIE aggregation	Provides the DEIE departmental distribution layer
SW-D-DCEE	Distribution	DCEE aggregation	Provides the DCEE departmental distribution layer
SW-D-DMME	Distribution	DMME aggregation	Provides the DMME departmental distribution layer
SW-A-DEIE	Access	DEIE end devices	Connects DEIE VPCs/end hosts
SW-A-DCEE	Access	DCEE end devices	Connects DCEE VPCs/end hosts
SW-A-DMME	Access	DMME end devices	Connects DMME VPCs/end hosts
SW-A-DIS	Access	DIS servers	Connects the server-zone devices

VM-AUTO	Automation	Netmiko, Ansible, SSH	Provides centralized automated configuration
VM-ZABBIX	Monitoring	Zabbix/SNMPv2c	Provides centralized network monitoring
VM-DHCP/WEB	Server	DHCP/DNS/Web	Provides network services and optional web reachability testing

3 ACL Policy Implementation

Access Control Lists (ACLs) were implemented to enforce the required security policies between the four departmental networks: DEIE, DCEE, DMME, and DIS. The ACLs were implemented as extended named ACLs. All the applied ACLs are attached in the Appendix.

Table 3-1 : ACL Deign According to Source and Destination

Source Network	Destination Network	Required Action
DEIE (VLAN 10)	DIS (VLAN 40)	Permit all
DCEE (VLAN 20)	DIS (VLAN 40)	Permit HTTP/HTTPS only
DMME (VLAN 30)	DIS (VLAN 40)	Deny all
DCEE (VLAN 20)	DEIE (VLAN 10)	Deny all
DMME (VLAN 30)	DCEE (VLAN 20)	Deny all
VLAN 99	All devices	Permit SSH (TCP 22)
Any	Any	Deny

3.1 4×4 reachability matrix

Source \ Destination	DEIE	DCEE	DMME	DIS
DEIE	—	deny	deny	PERMIT
DCEE	DENY	—	deny	HTTP/HTTPS ONLY

DMME	deny	DENY	—	DENY
DIS	permit	permit	permit	—

4 Network Automation

The Network Automation process was performed using the tools of Netmiko and Ansible that would help reduce the amount of manual configuration while sustaining consistency in the infrastructure. Netmiko was used for the R-EDGE and R-CORE routers while configuring using Ansible was performed in the Distribution and Access Switches. The main difference is that Netmiko uses procedural, python-based approach, implying that the user should define exact commands to be sent to the device, while Ansible uses declarative approach, where the user defines what configuration should be done on the target device using roles and variables.

4.1 Router Automation Using Netmiko

IP addresses, credentials, interface parameters, OSPF, NAT, ACL and SNMP information were stored in the inventory.yaml file. First of all, the YAML syntax was checked for validity. After that, common.py was written and it was used to load up the inventory, log, connect, catch exceptions and check the configuration. Lastly, the interface, OSPF, NAT parameters and VTY ACLs were generated on the devices using the configure_routers.py file and the information contained in the inventory.yaml file.

Before making any changes to the routers, the running configuration was pulled up to see the current configuration of the router. The code then matched the desired configuration with the current configuration of the routers. If there were no lines then the line had to be added with a chain of changes via the send_config_set() method in the Netmiko library. So, the script was idempotent, meaning that it wouldn't overwrite the configuration if it was already valid for the code. Changes to the configuration had been successful.

The push_snmp.py script loaded the target SNMP configuration, checked the current configuration, sent the needed but missing lines, and saved the configuration. The

connectivity, SNMP configuration, OSPF neighborship, and NAT configuration were tested by using the verify.py script, which gave either a “PASS” or “FAIL” for each test.

common.py - Provides SSH, logging, inventory, and configuration-checking

configure_routers.py - Configures router interfaces, OSPF, NAT, and VTY access.

inventory.yaml - Stores device IPs and router, SNMP, and management configuration.

push_snmp.py - Applies SNMP monitoring configuration to all devices.

verify.py - Verifies SSH, SNMP, OSPF, and NAT configurations after automation.

4.2 Switch Automation Using Ansible

As switch automation was needed, it was decided to use Ansible. Ansible was selected for switch automation due to the repetitive and structured configuration needs of the distribution and access switches. The project was based on an inventory with separate distribution and access groups, and group_vars defined common and group-specific parameters, host_vars were used to specify device-specific VLANs, trunk interfaces and access ports.

vlans, trunking, access_ports and stp are four open and reusable roles. departmental VLANs and common VLANs were configured by using cisco.ios.ios_vlans, trunk and access interfaces were configured by using cisco.ios.ios_l2_interfaces. The configured STP role enabled Rapid-PVST and PortFast and BPDU Guard on host-side access ports. This role-based structure minimized duplication of configuration and enhanced scalability.

Changes were orchestrated by using a site.yml file and were first previewed using the check mode of ansible (--check) and then executed using normal mode of the playbook. Another plus from the use of state: merged was that subsequent executions would not get unnecessarily reapplying already applied configuration. The idempotency evidence are attached at Appendix.

File structure and codes available in Git Hub : https://github.com/Asitha0012/Design-and-Management-of-Data-Networks

`group_vars/` - Group variables

`Access.yaml` - Enables BPDU Guard for access switches.

`All.yaml` - Stores common connection settings

`Distribution.yaml` - Disables BPDU Guard on distribution switches.

`host_vars/`

`SW-A-DCEE.yml`

`SW-A-DEIE.yml`

`SW-A-DIS.yml`

`SW-A-DMME.yml`

`SW-D-DCEE.yml`

`SW-D-DEIE.yml`

`SW-D-DMME.yml`

`Roles/vlans/tasks/main.yml` – Creates required VLANs.

`trunking/tasks/main.yml` – Configures trunk ports, VLANs

`access_ports/tasks/main.yml` – Assigns end-device ports to VLANs.

`stp/tasks/main.yml` – Configures Rapid-PVST, PortFast, and BPDU Guard

`ansible.cfg` - Defines Ansible execution settings and inventory location.

`inventory.ini` - Lists switches and their management IP addresses.

`site.yml` - Main playbook that runs VLAN, trunk, access-port, and STP

4.3 Tool Selection

Netmiko [1] and Ansible [2] were chosen because both tools use SSH-based connection to Cisco devices to support network automation and enable configurations be applied remotely without having to type commands on a device. Both tools can be used to automatically configure tasks that would otherwise be repetitive, minimize manual configuration errors and can help to validate the resulting device state. They can also be used with Python based automation environments, which is appropriate for this project's VM-AUTO environment, which is Python based.

There are two significant differences between the two tools, however, in how they automate. Netmiko takes a procedural approach, as python scripts specify what operations to be performed and what commands to be sent to the device, it gives fine granularity in device-specific configuration operations. In contrast, Ansible is a declarative model, where the desired configuration state is configured, and Ansible infers what changes are needed to reach the desired state. Ansible also has built-in support for a role-based structure that facilitates the reusability of configurations and the management and scaling of common configurations for many devices. For this reason, Netmiko was used for R-EDGE and R-CORE, gaining more control over the procedures used. The Netmiko scripts include interface configuration, OSPF, NAT, VTY access control, SNMP configuration, and check post-configuration. The idea was to use this way to make it possible to control the required router commands and verification using Python scripts.

Many of the distribution and access switches needed to be identical, and were distributed using ansible. Its roles and variables made deployment of VLANs, 802.1Q trunking, allow-VLAN lists, access-port assignments, and Rapid-PVST/BPDU Guard configurations easier. Information specific to the device was broken down into host variables and common settings were kept in group variables, which meant the same roles could be applied to multiple switches. So, due to these reasons Ansible was used to automate all the distribution and access switches. This was a perfect fit for fine-grained control for router automation and structured, scalable configuration management for switch automation.

5 Monitoring System

This section outlines how the proposed network infrastructure was designed and implemented to operate with the network monitoring system. The availability, performance and working state of network devices were continuously monitored by implementing a centralized monitoring solution with Zabbix [3] and SNMPv2c. Through the monitoring system, administrators can be alerted to problems in the network, see the parameters of network devices, and detect anomalies.

5.1 Zabbix setup

The monitoring architecture is made of a Zabbix monitoring server and several Cisco network devices. The Zabbix server was installed on an Ubuntu virtual machine and

routers and switches in the GNS3 topology were set up as routers/switches agents with SNMP protocol. The communication between the monitoring server and the network devices were enabled with SNMPv2c protocol.

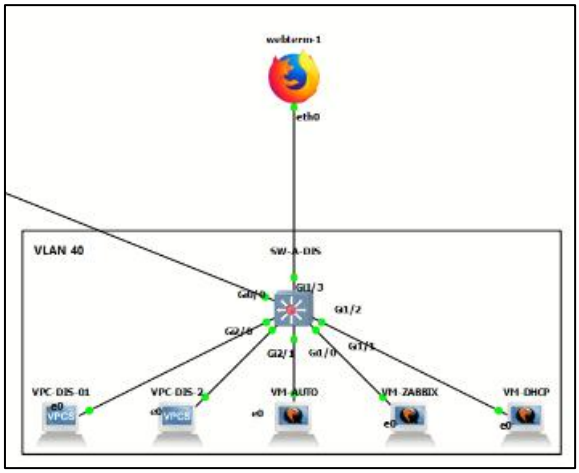

Figure 5-1 ; Zabbix setup with Web

5.2 Zabbix Server Configuration

The Zabbix monitoring server was installed on an Ubuntu virtual machine located in the management VLAN. The required services including Apache web server, database server, and Zabbix server services were configured to provide the monitoring platform.

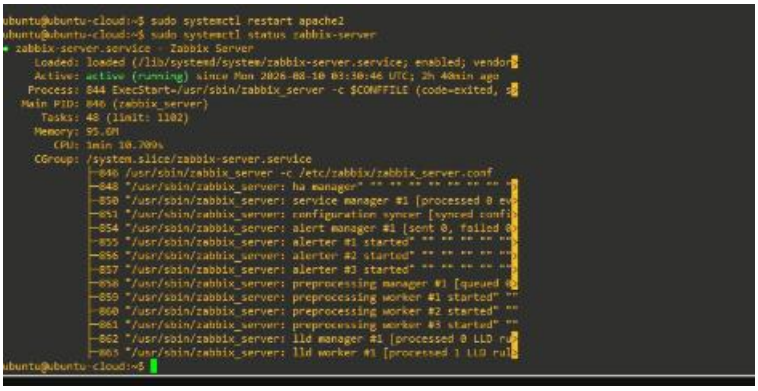

Figure 5-2 : Logs of Zabbix Server Up and Running

5.3 Host list

Cisco devices were added to the Zabbix monitoring platform as hosts. Each device was configured with Device IP address, SNMP interface, SNMPV2c community String and Cisco IOS SNMP monitoring template.

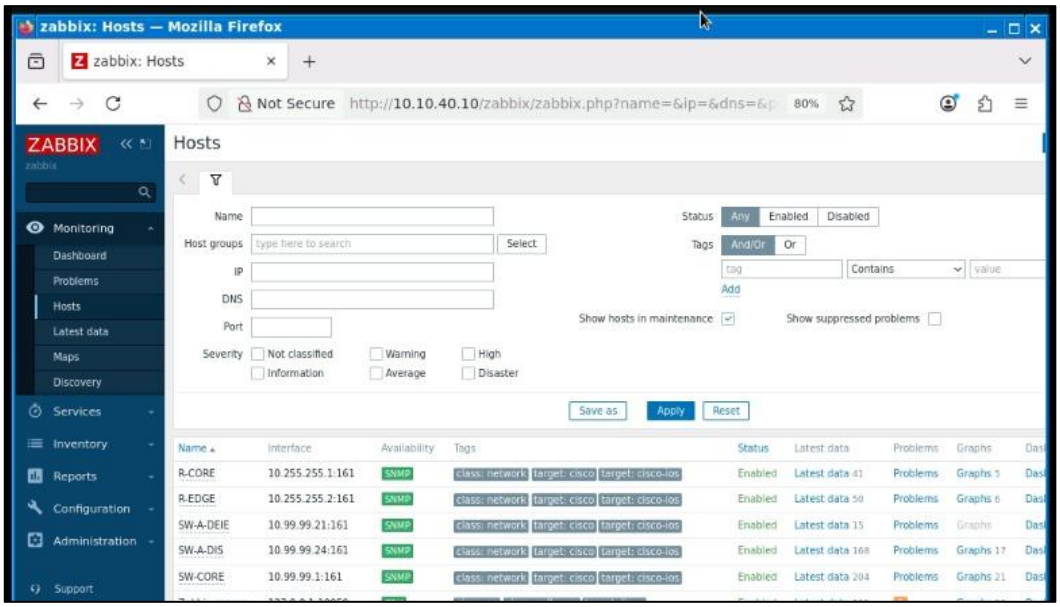

Figure 5-3 : List of Hosts of Zabbix Dashboard

5.4 Triggers

Triggers were configured in Zabbix to automatically generate alerts when predefined conditions occur. Implemented triggers include:

- Device unreachable due to ICMP timeout
- Network interface down condition
- High CPU utilization

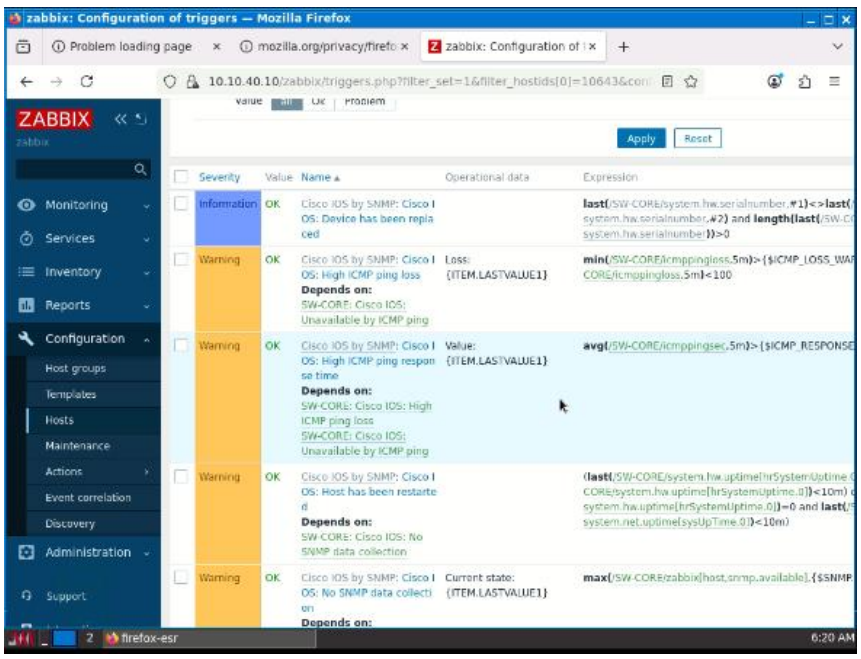

Table 5-1 : Dashboard UI of Custom Monitoring Triggers

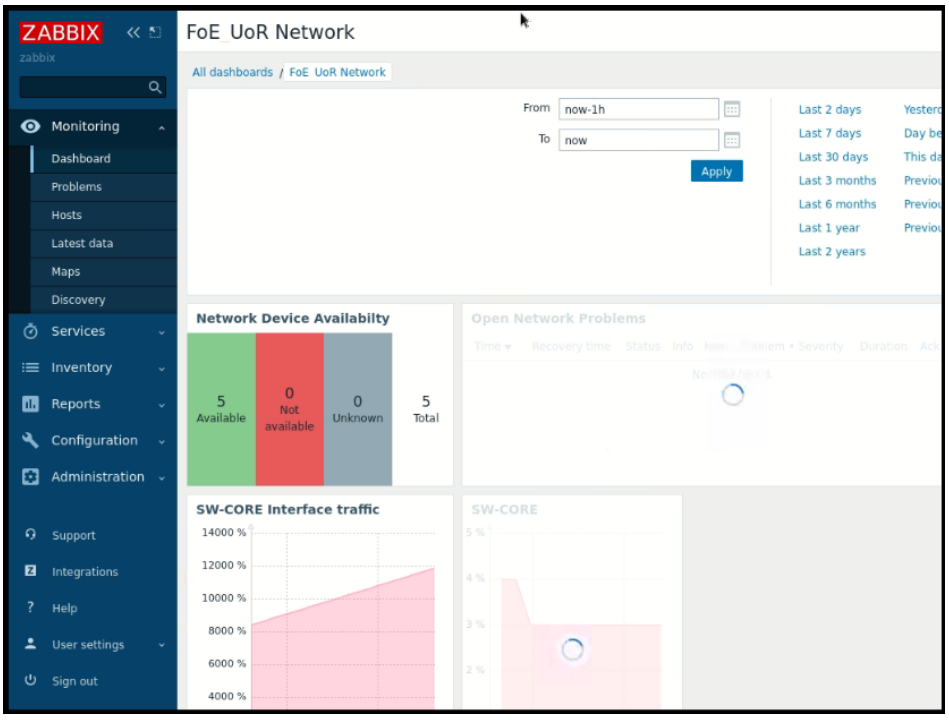

Figure 5-4 : Zabbix Dashboaoard Descriptions

The rolled out Zabbix monitoring system made it possible to obtain centralized monitoring of the network infrastructure successfully. Continuous monitoring for availability, interface status and performance parameters was accomplished through SNMPv2c integration with Cisco devices. The alerting mechanism enhanced the fault detection capability and proved the effectiveness of centralized network monitoring in enterprise network environment.

6 Challenges and Lessons Learned

There were some major hurdles to implementing the campus network. Care was taken to configure VLAN and trunk between distribution switches and access switches: Each department got its own VLAN, VLAN 99 was set up to operate as a network management VLAN, and VLAN 100 was configured as native. It was critical to make sure that the proper VLANs were allowed on each trunk.

A major issue that we faced was the ‘ip routing’ enabled in layer 2 access switches and that caused to ignore the ‘ip default-gateway’ command. The remedy was disabling the IP routing on the access switches to configure them into Layer 2. Another issue was the configuration of the inter-VLAN routing and the OSPF protocol to advertise the routes from R-EDGE, R-CORE and SW-CORE through OSPF area 0. Default static routes (SW-CORE->R-CORE->R-EDGE) had to be configured to reach the Internet,

which was later validated in an end-to-end test. The DHCP/DNS had to be configured, which entailed multiple VLANs serving, dhcp relay feature enabled on SW-CORE to forward the dhcp discover-requests to the central server, and the successful test of the dhcp on VLANs 10,20,30.

The automation part of the project required the creation of the network using Netmiko and Ansible which had multiple pitfalls, including the inventory file, ip addresses, ssh access, usernames/passwords and the compatible ssh configuration for the Cisco switches to be correctly discovered.

The project provided the insight into the processes of designing, implementing and managing enterprise-level network and its key components such as VLANs, routing, ACLs, SNMP, Zabbix and Ansible. The network planning, the IP subnetting and the configuration in particular were thoroughly learned throughout the project development. The experience of creating the ACLs deepened the understanding of the network security fields, particularly the packet filtering and troubleshooting mechanisms. In addition, the Zabbix and SNMP utilization provided the essential knowledge and experience in the network monitoring and management area. Similarly, the Ansible-based automation of the network configuration and management tasks introduced the main benefits of the automation in network configuration and management, including the minimized human intervention and thus the errors and increased efficiency. Overall, the project has enhanced the practical skills in the network designing, security, monitoring and automation areas. Finally, the project also emphasized the convergence of the network technologies.

7 References

[1] “Module netmiko - Python for network engineers.” Accessed: Aug. 10, 2026. [Online]. Available: https://pyneng.readthedocs.io/en/latest/book/18_ssh_telnet/netmiko.html

[2] “Ansible Documentation — Ansible Community Documentation,” 2025. Accessed: Aug. 10, 2026. [Online]. Available: https://docs.ansible.com/projects/ansible/latest/index.html

[3] “Zabbix Manual.” Accessed: Aug. 10, 2026. [Online]. Available: https://www.zabbix.com/documentation/current/en/manual

8 Appendix A

(Git Hub : https://github.com/Asitha0012/Design-and-Management-of-Data-Networks)

Support evidence for Section 3

ACL 4X4 Matrix evidences

```
    100 deny ip 10.10.20.0 0.0.0.255 10.0.0.0 0.255.255.255 log
    110 permit ip 10.10.20.0 0.0.0.255 any
    120 deny ip any any log
Extended IP access list ACL_DEIE_IN
    10 permit udp any eq bootpc any eq bootps
    20 permit icmp 10.10.10.0 0.0.0.255 host 10.10.10.1 echo
    30 permit ip 10.10.10.0 0.0.0.255 10.10.40.0 0.0.0.255 (10 matches)
    40 deny ip 10.10.10.0 0.0.0.255 10.10.20.0 0.0.0.255 log (5 matches)
    50 deny ip 10.10.10.0 0.0.0.255 10.10.30.0 0.0.0.255 log (5 matches)
    60 deny ip 10.10.10.0 0.0.0.255 10.99.99.0 0.0.0.255 log
    70 deny ip 10.10.10.0 0.0.0.255 10.0.0.0 0.255.255.255 log
    80 permit ip 10.10.10.0 0.0.0.255 any
    90 deny ip any any log
Extended IP access list ACL_DIS_IN
    10 permit udp host 10.10.40.10 any eq snmp
    20 permit icmp host 10.10.40.10 any
    30 permit udp host 10.10.40.20 eq bootps any
    40 permit udp host 10.10.40.20 eq domain any
    50 permit tcp 10.10.40.0 0.0.0.255 any established
    60 permit icmp 10.10.40.0 0.0.0.255 any echo-reply
    70 permit icmp 10.10.40.0 0.0.0.255 any port-unreachable
    80 deny ip any any log
Extended IP access list ACL_DMME_IN
    10 permit udp any eq bootpc any eq bootps
    20 permit icmp 10.10.30.0 0.0.0.255 host 10.10.30.1 echo
    30 deny ip 10.10.30.0 0.0.0.255 10.10.40.0 0.0.0.255 log (10 matches)
    40 deny ip 10.10.30.0 0.0.0.255 10.10.20.0 0.0.0.255 log (5 matches)
    50 deny ip 10.10.30.0 0.0.0.255 10.10.10.0 0.0.0.255 log (5 matches)
    60 deny ip any any log
Extended IP access list ACL_MGMT_IN
    10 permit tcp 10.99.99.0 0.0.0.255 any eq 22
    20 permit icmp 10.99.99.0 0.0.0.255 any
    30 permit tcp 10.99.99.0 0.0.0.255 host 10.10.40.10 eq www
    40 permit udp 10.99.99.0 0.0.0.255 eq snmp host 10.10.40.10
    50 permit udp 10.99.99.0 0.0.0.255 host 10.10.40.10 eq snmptrap
    60 deny ip any any log
Extended IP access list CISCO-CWA-URL-REDIRECT-ACL
    100 deny udp any any eq domain
    101 deny tcp any any eq domain
    102 deny udp any eq bootps any
    103 deny udp any any eq bootpc
    104 deny udp any eq bootpc any
    105 permit tcp any any eq www
Extended IP access list preauth_ipv4_acl (per-user)
    10 permit udp any any eq domain
    20 permit tcp any any eq domain
    30 permit udp any eq bootps any
    40 permit udp any any eq bootpc
    50 permit udp any eq bootpc any
    60 deny ip any any
IPv6 access list preauth_ipv6_acl (per-user)
    permit udp any any eq domain sequence 10
    permit tcp any any eq domain sequence 20
    permit icmp any any nd-ns sequence 30
    permit icmp any any nd-na sequence 40
    permit icmp any any router-solicitation sequence 50
    permit icmp any any router-advertisement sequence 60
    permit icmp any any redirect sequence 70
    permit udp any eq 547 any eq 546 sequence 80
    permit udp any eq 546 any eq 547 sequence 90
    deny ipv6 any any sequence 100
SW-CORE>
SW-CORE>
```

Figure 8-2 : Applied Extended Access-Lists

```
VPC-DIS-01> show ip

NAME        : VPC-DIS-01[1]
IP/MASK     : 10.10.40.51/24
GATEWAY     : 10.10.40.1
DNS         :
MAC         : 00:50:79:66:68:07
LPORT       : 20330
RHOST:PORT  : 127.0.0.1:20331
MTU         : 1500

VPC-DIS-01> ping 10.10.10.51

84 bytes from 10.10.10.51 icmp_seq=1 ttl=63 time=38.523 ms
84 bytes from 10.10.10.51 icmp_seq=2 ttl=63 time=21.112 ms
84 bytes from 10.10.10.51 icmp_seq=3 ttl=63 time=19.799 ms
84 bytes from 10.10.10.51 icmp_seq=4 ttl=63 time=14.025 ms
84 bytes from 10.10.10.51 icmp_seq=5 ttl=63 time=19.694 ms

VPC-DIS-01> ping 10.10.10.51

84 bytes from 10.10.10.51 icmp_seq=1 ttl=63 time=23.934 ms
84 bytes from 10.10.10.51 icmp_seq=2 ttl=63 time=17.526 ms
84 bytes from 10.10.10.51 icmp_seq=3 ttl=63 time=17.745 ms
84 bytes from 10.10.10.51 icmp_seq=4 ttl=63 time=16.463 ms
84 bytes from 10.10.10.51 icmp_seq=5 ttl=63 time=19.001 ms

VPC-DIS-01> ping 10.10.10.51

84 bytes from 10.10.10.51 icmp_seq=1 ttl=63 time=33.913 ms
84 bytes from 10.10.10.51 icmp_seq=2 ttl=63 time=19.383 ms
```

Figure 8-1 : Results of ACL Test from DIS

```
VPC-DEIE-01>
VPC-DEIE-01>
VPC-DEIE-01>  ip 10.10.10.51 255.255.255.0 10.10.10.1
Checking for duplicate address...
VPC-DEIE-01 : 10.10.10.51 255.255.255.0 gateway 10.10.10.1

VPC-DEIE-01> show ip

NAME        : VPC-DEIE-01[1]
IP/MASK     : 10.10.10.51/24
GATEWAY     : 10.10.10.1
DNS         : 10.10.40.20
DHCP SERVER : 10.10.40.20
DHCP LEASE  : 43140, 43200/21600/37800
DOMAIN NAME : foe.uor.lk
MAC         : 00:50:79:66:68:06
LPORT       : 20328
RHOST:PORT  : 127.0.0.1:20329
MTU         : 1500

VPC-DEIE-01>
VPC-DEIE-01>
VPC-DEIE-01>
VPC-DEIE-01> ping 10.10.20.51

*10.10.10.1 icmp_seq=1 ttl=255 time=21.482 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=2 ttl=255 time=19.906 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=3 ttl=255 time=16.417 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=4 ttl=255 time=16.798 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=5 ttl=255 time=19.141 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DEIE-01> ping 10.10.30.51

*10.10.10.1 icmp_seq=1 ttl=255 time=16.931 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=2 ttl=255 time=16.313 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=3 ttl=255 time=14.985 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=4 ttl=255 time=17.710 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=5 ttl=255 time=14.644 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DEIE-01> ping 10.10.40.51

10.10.40.51 icmp_seq=1 timeout
84 bytes from 10.10.40.51 icmp_seq=2 ttl=63 time=35.469 ms
84 bytes from 10.10.40.51 icmp_seq=3 ttl=63 time=21.989 ms
84 bytes from 10.10.40.51 icmp_seq=4 ttl=63 time=18.029 ms
84 bytes from 10.10.40.51 icmp_seq=5 ttl=63 time=15.800 ms

VPC-DEIE-01>
```

Figure 8-4 : Results of ACL Test from DEIE

```
NAME        : VPC-DMME-01[1]
IP/MASK     : 10.10.30.51/24
GATEWAY     : 10.10.30.1
DNS         :
MAC         : 00:50:79:66:68:04
LPORT       : 20324
RHOST:PORT  : 127.0.0.1:20325
MTU         : 1500

VPC-DMME-01> ping 10.10.10.51

*10.10.30.1 icmp_seq=1 ttl=255 time=16.554 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=2 ttl=255 time=16.430 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=3 ttl=255 time=21.855 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=4 ttl=255 time=21.322 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=5 ttl=255 time=15.467 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DMME-01> ping 10.10.20.51

*10.10.30.1 icmp_seq=1 ttl=255 time=15.654 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=2 ttl=255 time=15.271 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=3 ttl=255 time=16.220 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=4 ttl=255 time=15.185 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=5 ttl=255 time=14.867 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DMME-01> ping 10.10.40.51

*10.10.30.1 icmp_seq=1 ttl=255 time=27.759 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=2 ttl=255 time=16.081 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=3 ttl=255 time=33.747 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=4 ttl=255 time=26.514 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=5 ttl=255 time=18.698 ms (ICMP type:3, code:13, Communication administratively prohibited)
```

Figure 8-3 : Results of ACL of from DMME

```
NAME        : VPC-DCEE-01[1]
IP/MASK     : 0.0.0.0/0
GATEWAY     : 0.0.0.0
DNS         :
MAC         : 00:50:79:66:68:02
LPORT       : 20320
RHOST:PORT  : 127.0.0.1:20321
MTU         : 1500

VPC-DCEE-01> ip 10.10.20.51 255.255.255.0 10.10.20.1
Checking for duplicate address...
VPC-DCEE-01 : 10.10.20.51 255.255.255.0 gateway 10.10.20.1

VPC-DCEE-01> save
Saving startup configuration to startup.vpc
.  done

VPC-DCEE-01> ping 10.10.10.51

*10.10.20.1 icmp_seq=1 ttl=255 time=16.824 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=44.784 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=3 ttl=255 time=15.737 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=4 ttl=255 time=15.839 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=5 ttl=255 time=14.264 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DCEE-01> ping 10.10.30.51

*10.10.20.1 icmp_seq=1 ttl=255 time=28.436 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=14.645 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=3 ttl=255 time=16.443 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=4 ttl=255 time=16.255 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=5 ttl=255 time=19.324 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DCEE-01> ping 10.10.40.51

*10.10.20.1 icmp_seq=1 ttl=255 time=19.251 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=14.638 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=3 ttl=255 time=19.959 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=4 ttl=255 time=16.896 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=5 ttl=255 time=17.118 ms (ICMP type:3, code:13, Communication administratively prohibited)

VPC-DCEE-01>
```

Figure 8-5 : Results of ACL test from DCEE

Support Evidence for Section 4 (Automation)

```
▯[?2004hubuntu@ubuntu-cloud:~/netmiko$ mpython3 configure_routers.py
▯[?2004l
2026-08-09 08:14:58,736 - INFO    - Log file: /home/ubuntu/netmiko/logs/configure_routers_20260809_081458.log
2026-08-09 08:14:58,748 - INFO    - [R-CORE] connecting to 10.255.255.1...
2026-08-09 08:14:59,831 - INFO    - [R-CORE] connected
2026-08-09 08:15:01,983 - INFO    - [R-CORE] pushing 4 line(s):
2026-08-09 08:15:06,171 - INFO    - [R-CORE] configuration saved (write memory)
2026-08-09 08:15:06,195 - INFO    - [R-EDGE] connecting to 10.255.255.2...
2026-08-09 08:15:07,251 - INFO    - [R-EDGE] connected
2026-08-09 08:15:09,482 - INFO    - [R-EDGE] pushing 12 line(s):
2026-08-09 08:15:19,925 - INFO    - [R-EDGE] configuration saved (write memory)
2026-08-09 08:15:19,959 - INFO    - ==== SUMMARY ====
2026-08-09 08:15:19,961 - INFO    - R-CORE  : ok
2026-08-09 08:15:19,962 - INFO    - R-EDGE  : ok
▯[?2004hubuntu@ubuntu-cloud:~/netmiko$
▯[?2004l
▯[?2004hubuntu@ubuntu-cloud:~/netmiko$ ^C▯[?2004l
▯[?2004h▯[?2004l
▯[?2004hubuntu@ubuntu-cloud:~/netmiko$ ▯[7mpython3 push_snmp.py▯[27m▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯python3 push_snmp.py
▯[?2004l
2026-08-09 08:18:15,317 - INFO    - Log file: /home/ubuntu/netmiko/logs/push_snmp_20260809_081815.log
2026-08-09 08:18:15,328 - INFO    - [R-CORE] connecting to 10.255.255.1...
2026-08-09 08:18:16,394 - INFO    - [R-CORE] connected
2026-08-09 08:18:18,534 - INFO    - [R-CORE] pushing 5 snmp-server line(s)
2026-08-09 08:18:23,061 - INFO    - [R-CORE] configuration saved
2026-08-09 08:18:23,084 - INFO    - [R-EDGE] connecting to 10.255.255.2...
2026-08-09 08:18:23,926 - INFO    - [R-EDGE] connected
2026-08-09 08:18:26,160 - INFO    - [R-EDGE] pushing 5 snmp-server line(s)
2026-08-09 08:18:31,219 - INFO    - [R-EDGE] configuration saved
2026-08-09 08:18:31,243 - INFO    - [SW-CORE] connecting to 10.99.99.1...
2026-08-09 08:18:32,236 - INFO    - [SW-CORE] connected
2026-08-09 08:18:36,802 - INFO    - [SW-CORE] pushing 5 snmp-server line(s)
2026-08-09 08:18:45,342 - INFO    - [SW-CORE] configuration saved
2026-08-09 08:18:45,419 - INFO    - [SW-D-DEIE] connecting to 10.99.99.11...
2026-08-09 08:18:46,549 - INFO    - [SW-D-DEIE] connected
2026-08-09 08:18:50,398 - INFO    - [SW-D-DEIE] pushing 5 snmp-server line(s)
2026-08-09 08:18:59,547 - INFO    - [SW-D-DEIE] configuration saved
2026-08-09 08:18:59,591 - INFO    - [SW-D-DCEE] connecting to 10.99.99.12...
2026-08-09 08:19:00,682 - INFO    - [SW-D-DCEE] connected
```

Figure 8-6 : Netmiko SNMP Execution Logs (I)

```
2026-08-09 08:18:36,802 - INFO    - [SW-CORE] pushing 5 snmp-server line(s)
2026-08-09 08:18:45,342 - INFO    - [SW-CORE] configuration saved
2026-08-09 08:18:45,419 - INFO    - [SW-D-DEIE] connecting to 10.99.99.11...
2026-08-09 08:18:46,549 - INFO    - [SW-D-DEIE] connected
2026-08-09 08:18:50,398 - INFO    - [SW-D-DEIE] pushing 5 snmp-server line(s)
2026-08-09 08:18:59,547 - INFO    - [SW-D-DEIE] configuration saved
2026-08-09 08:18:59,591 - INFO    - [SW-D-DCEE] connecting to 10.99.99.12...
2026-08-09 08:19:00,682 - INFO    - [SW-D-DCEE] connected
2026-08-09 08:19:03,800 - INFO    - [SW-D-DCEE] pushing 5 snmp-server line(s)
2026-08-09 08:19:12,033 - INFO    - [SW-D-DCEE] configuration saved
2026-08-09 08:19:12,077 - INFO    - [SW-D-DMME] connecting to 10.99.99.13...
2026-08-09 08:19:13,110 - INFO    - [SW-D-DMME] connected
2026-08-09 08:19:16,234 - INFO    - [SW-D-DMME] pushing 5 snmp-server line(s)
2026-08-09 08:19:25,203 - INFO    - [SW-D-DMME] configuration saved
2026-08-09 08:19:25,251 - INFO    - [SW-A-DEIE] connecting to 10.99.99.21...
2026-08-09 08:19:28,515 - INFO    - [SW-A-DEIE] connected
2026-08-09 08:19:32,347 - INFO    - [SW-A-DEIE] pushing 5 snmp-server line(s)
2026-08-09 08:19:40,270 - INFO    - [SW-A-DEIE] configuration saved
2026-08-09 08:19:40,303 - INFO    - [SW-A-DCEE] connecting to 10.99.99.22...
2026-08-09 08:19:43,507 - INFO    - [SW-A-DCEE] connected
2026-08-09 08:19:47,270 - INFO    - [SW-A-DCEE] pushing 5 snmp-server line(s)
2026-08-09 08:19:55,646 - INFO    - [SW-A-DCEE] configuration saved
2026-08-09 08:19:55,695 - INFO    - [SW-A-DMME] connecting to 10.99.99.23...
2026-08-09 08:19:57,057 - INFO    - [SW-A-DMME] connected
2026-08-09 08:20:00,763 - INFO    - [SW-A-DMME] pushing 5 snmp-server line(s)
2026-08-09 08:20:08,464 - INFO    - [SW-A-DMME] configuration saved
2026-08-09 08:20:08,508 - INFO    - [SW-A-DIS] connecting to 10.99.99.24...
2026-08-09 08:20:09,423 - INFO    - [SW-A-DIS] connected
2026-08-09 08:20:13,214 - INFO    - [SW-A-DIS] pushing 5 snmp-server line(s)
2026-08-09 08:20:22,259 - INFO    - [SW-A-DIS] configuration saved
2026-08-09 08:20:22,284 - INFO    - ==== SUMMARY (10 devices) ====
2026-08-09 08:20:22,287 - INFO    -  R-CORE    : ok
2026-08-09 08:20:22,289 - INFO    -  R-EDGE    : ok
2026-08-09 08:20:22,298 - INFO    -  SW-CORE   : ok
2026-08-09 08:20:22,303 - INFO    -  SW-D-DEIE : ok
2026-08-09 08:20:22,306 - INFO    -  SW-D-DCEE : ok
2026-08-09 08:20:22,308 - INFO    -  SW-D-DMME : ok
2026-08-09 08:20:22,310 - INFO    -  SW-A-DEIE : ok
2026-08-09 08:20:22,312 - INFO    -  SW-A-DCEE : ok
2026-08-09 08:20:22,314 - INFO    -  SW-A-DMME : ok
2026-08-09 08:20:22,316 - INFO    -  SW-A-DIS  : ok
```

Figure 8-7 : Netmiko SNMP Execution Logs (II)

```
=N=N=N=N=N=N=N=N=N=N=N=N= PuTTY log 2026.08.09 16:48:52 =N=N=N=N=N=N=N=N=N=N=N=N=

[?2004l
[?2004hubuntu@ubuntu-cloud:~$ cd netmiko
[?2004l
[?2004hubuntu@ubuntu-cloud:~/netmiko$ ls
[?2004l
__pycache__  configure_routers.py  logs          requirements.txt
common.py    inventory.yaml        push_snmp.py  verify.py
[?2004hubuntu@ubuntu-cloud:~/netmiko$ py t hon3 verify.py
[?2004l
2026-08-09 11:19:18,885 - INFO    - Log file: /home/ubuntu/netmiko/logs/verify_20260809_111918.log
2026-08-09 11:19:18,897 - INFO    - DEVICE     CHECK                RESULT
2026-08-09 11:19:18,899 - INFO    - [R-CORE] connecting to 10.255.255.1...
2026-08-09 11:19:19,927 - INFO    - [R-CORE] connected
2026-08-09 11:19:22,338 - INFO    - R-CORE     ssh-reachable        PASS
2026-08-09 11:19:22,340 - INFO    - R-CORE     snmp-community       PASS
2026-08-09 11:19:22,342 - INFO    - R-CORE     ospf-neighbor-full   PASS
2026-08-09 11:19:22,347 - INFO    - [R-EDGE] connecting to 10.255.255.2...
2026-08-09 11:19:23,404 - INFO    - [R-EDGE] connected
2026-08-09 11:19:26,096 - INFO    - R-EDGE     ssh-reachable        PASS
2026-08-09 11:19:26,099 - INFO    - R-EDGE     snmp-community       PASS
2026-08-09 11:19:26,100 - INFO    - R-EDGE     ospf-neighbor-full   PASS
2026-08-09 11:19:26,102 - INFO    - R-EDGE     nat-configured       PASS
2026-08-09 11:19:26,105 - INFO    - [SW-CORE] connecting to 10.99.99.1...
2026-08-09 11:19:27,158 - INFO    - [SW-CORE] connected
2026-08-09 11:19:31,963 - INFO    - SW-CORE    ssh-reachable        PASS
2026-08-09 11:19:31,966 - INFO    - SW-CORE    snmp-community       PASS
2026-08-09 11:19:31,968 - INFO    - [SW-D-DEIE] connecting to 10.99.99.11...
2026-08-09 11:19:32,994 - INFO    - [SW-D-DEIE] connected
2026-08-09 11:19:36,797 - INFO    - SW-D-DEIE  ssh-reachable        PASS
2026-08-09 11:19:36,799 - INFO    - SW-D-DEIE  snmp-community       PASS
2026-08-09 11:19:36,802 - INFO    - [SW-D-DCEE] connecting to 10.99.99.12...
2026-08-09 11:19:37,837 - INFO    - [SW-D-DCEE] connected
2026-08-09 11:19:41,870 - INFO    - SW-D-DCEE  ssh-reachable        PASS
2026-08-09 11:19:41,874 - INFO    - SW-D-DCEE  snmp-community       PASS
2026-08-09 11:19:41,877 - INFO    - [SW-D-DMME] connecting to 10.99.99.13...
2026-08-09 11:19:42,924 - INFO    - [SW-D-DMME] connected
2026-08-09 11:19:46,772 - INFO    - SW-D-DMME  ssh-reachable        PASS
2026-08-09 11:19:46,774 - INFO    - SW-D-DMME  snmp-community       PASS
2026-08-09 11:19:46,777 - INFO    - [SW-A-DEIE] connecting to 10.99.99.21...
2026-08-09 11:19:47,781 - INFO    - [SW-A-DEIE] connected
2026-08-09 11:19:51,264 - INFO    - SW-A-DEIE  ssh-reachable        PASS
2026-08-09 11:19:51,267 - INFO    - SW-A-DEIE  snmp-community       PASS
2026-08-09 11:19:51,269 - INFO    - [SW-A-DCEE] connecting to 10.99.99.22...
2026-08-09 11:19:52,385 - INFO    - [SW-A-DCEE] connected
2026-08-09 11:19:55,430 - INFO    - SW-A-DCEE  ssh-reachable        PASS
2026-08-09 11:19:55,433 - INFO    - SW-A-DCEE  snmp-community       PASS
2026-08-09 11:19:55,434 - INFO    - [SW-A-DMME] connecting to 10.99.99.23...
2026-08-09 11:19:56,531 - INFO    - [SW-A-DMME] connected
2026-08-09 11:20:00,332 - INFO    - SW-A-DMME  ssh-reachable        PASS
2026-08-09 11:20:00,335 - INFO    - SW-A-DMME  snmp-community       PASS
2026-08-09 11:20:00,337 - INFO    - [SW-A-DIS] connecting to 10.99.99.24...
2026-08-09 11:20:01,270 - INFO    - [SW-A-DIS] connected
2026-08-09 11:20:05,050 - INFO    - SW-A-DIS   ssh-reachable        PASS
2026-08-09 11:20:05,052 - INFO    - SW-A-DIS   snmp-community       PASS
2026-08-09 11:20:05,054 - INFO    - ==== OVERALL: PASS ====
[?2004hubuntu@ubuntu-cloud:~/netmiko$
```

Figure 8-8 : Netmiko Verification Logs

```
PLAY [Configure EE8203 distribution and access switches] **********************
[WARNING]: Collection ansible.netcommon does not support Ansible version 2.10.8

TASK [vlans : Ensure common and department VLANs exist] ***********************
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
ok: [SW-A-DEIE]
ok: [SW-D-DMME]
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-D-DEIE]
ok: [SW-D-DCEE]
ok: [SW-A-DCEE]
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
[WARNING]: ansible-pylibssh not installed, falling back to paramiko
ok: [SW-A-DMME]
ok: [SW-A-DIS]

TASK [trunking : Ensure trunk ports are 802.1Q trunks with pruned VLAN lists] ***
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-A-DEIE] => (item=GigabitEthernet0/0)
ok: [SW-D-DMME] => (item=GigabitEthernet0/0)
ok: [SW-A-DCEE] => (item=GigabitEthernet0/0)
ok: [SW-D-DCEE] => (item=GigabitEthernet0/0)
ok: [SW-D-DEIE] => (item=GigabitEthernet0/0)
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-D-DEIE] => (item=GigabitEthernet0/1)
```

Figure 8-9 : Ansible Pre-Check Logs

```
TASK [stp : Enable PortFast BPDU guard on host-facing ports] ******************
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-A-DEIE] => (item={'name': 'GigabitEthernet1/0', 'description': 'VPC-DEIE-01', 'vlan': 10})
ok: [SW-A-DCEE] => (item={'name': 'GigabitEthernet1/0', 'description': 'VPC-DCEE-01', 'vlan': 20})
ok: [SW-A-DEIE] => (item={'name': 'GigabitEthernet1/1', 'description': 'VPC-DEIE-02', 'vlan': 10})
ok: [SW-A-DCEE] => (item={'name': 'GigabitEthernet1/1', 'description': 'VPC-DCEE-02', 'vlan': 20})
ok: [SW-A-DMME] => (item={'name': 'GigabitEthernet1/0', 'description': 'VPC-DMME-01', 'vlan': 30})
ok: [SW-A-DMME] => (item={'name': 'GigabitEthernet1/1', 'description': 'VPC-DMME-02', 'vlan': 30})
ok: [SW-A-DIS] => (item={'name': 'GigabitEthernet1/0', 'description': 'VM-AUTO', 'vlan': 99})
ok: [SW-A-DIS] => (item={'name': 'GigabitEthernet1/1', 'description': 'VM-ZABBIX', 'vlan': 40})
ok: [SW-A-DIS] => (item={'name': 'GigabitEthernet1/2', 'description': 'VM-DHCP', 'vlan': 40})
ok: [SW-A-DIS] => (item={'name': 'GigabitEthernet2/0', 'description': 'VPC-DIS-01', 'vlan': 40})
ok: [SW-A-DIS] => (item={'name': 'GigabitEthernet2/1', 'description': 'VPC-DIS-02', 'vlan': 40})

TASK [Persist configuration to startup-config] ********************************
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-A-DEIE]
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-D-DMME]
[WARNING]: Collection ansible.utils does not support Ansible version 2.10.8
ok: [SW-A-DCEE]
ok: [SW-D-DCEE]
ok: [SW-D-DEIE]
ok: [SW-A-DMME]
ok: [SW-A-DIS]

PLAY RECAP *********************************************************************
SW-A-DCEE                  : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW-A-DEIE                  : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW-A-DIS                   : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW-A-DMME                  : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
SW-D-DCEE                  : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
SW-D-DEIE                  : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
SW-D-DMME                  : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0

[?2004hubuntu@ubuntu-cloud:~/ansible$
```

Figure 8-10 : Ansible Idempotency Verification (Changed = 0)